mdstats 3-D Graphics User Guide

Practical configurable framework, connectivity, trajectory, and density visualization with mdstats-3d

mdstats user guide

2026-08-11

Contents

What `mdstats-3d` does

`mdstats-3d` is the universal configurable 3-D scene command introduced in `mdstats 0.20.148a0`; `0.20.149a0` adds the shared scientific dependency DAG, `0.20.150a0` completes renderer-neutral composition/view semantics, `0.20.156a0` hardens topology reuse and long-trajectory framework preparation, and `0.20.157a0` hardens failure handling, sparse-density rendering, streaming LAMMPS selection, shared connectivity geometry, and density-plan reuse; `0.20.158a0` hardens long-trajectory connectivity/MIC handling and accelerates atomic mean-graph preparation; `0.20.159a0` makes direct sparse density realization consume PAR-DENS worker leases and adds field/kernel progress reporting.

The first four independent graphical layer types are:

```
framework      zeolite/framework topology
connectivity   atomic mean connectivity
trajectory     selected atomic trajectories
density        selected atomic probability density
```

You may display any combination and may use more than one instance of the same type.

Source-tree and installed commands

From an mdstats source checkout:

```
python tools/mdstats-3d.py TRAJECTORY ...
```

After package installation:

```
mdstats-3d TRAJECTORY ...
```

Fast layer selection

Framework only:

```
mdstats-3d dump.lammpstrj \
  --lammps-units metal \
  --lammps-timestep 0.001 \
  --lammps-type-map "1=Si,2=Al,3=O,4=Na" \
  --layer framework
```

Framework plus Na trajectories:

```
mdstats-3d dump.lammpstrj \
  --lammps-units metal \
  --lammps-timestep 0.001 \
  --lammps-type-map "1=Si,2=Al,3=O,4=Na" \
  --layer framework \
  --layer trajectory:Na
```

Na-O connectivity plus Na density:

```
mdstats-3d dump.lammpstrj \
  --lammps-units metal \
  --lammps-timestep 0.001 \
  --lammps-type-map "1=Si,2=Al,3=O,4=Na" \
  --layer connectivity:Na-O \
  --layer density:Na
```

Give a layer a custom name with `@`:

```
--layer density:Na@mobile-Na-density
```

Reproduce the historical LTA hybrid plot

The former `examples/plot_lta_mixed_alkali_density.py` workflow is now a preset:

```
mdstats-3d dump.lammpstrj \
  --preset lta-mixed-alkali-density \
  --lammps-units metal \
  --lammps-timestep 0.001 \
  --lammps-type-map "1=Si,2=Al,3=O,4=Na"
```

The preset detects which of Si, Al, O, Li, Na, and K are present and builds framework, atomic connectivity, trajectories, and one density layer per present supported species.

The old example script still works as a compatibility launcher.

Use TOML for publication scenes

For a reusable figure, prefer TOML over a long shell command.

Example `na_lta.toml`:

```
[scene]
title = "Na-LTA 300 K"
registration = "framework_registered"
display_cell = "reference"
trajectory_display_mode = "folded"
density_grid_interval = 0.20
density_adaptive_smearing = true
projection = "orthographic"

[input]
format = "lammps-dump"
lammps_units = "metal"
lammps_timestep = 0.001
```

```toml
lammps_type_map = "1=Si,2=Al,3=O,4=Na"

[resources]
max_threads = 8
max_memory = "12GiB"

[output]
path = "na_lta.html"
manifest = "na_lta.scene.json"
browser_profile = "balanced"

[[layer]]
type = "framework"
name = "LTA framework"

[[layer]]
type = "connectivity"
name = "Na-O coordination"
selection = { pairs = ["Na-O"] }
render = { edge_width = 1.6, edge_opacity = 0.45 }

[[layer]]
type = "trajectory"
name = "Na trajectories"
selection = { species = ["Na"] }
render = { line_width = 1.6, opacity = 0.28, show_legend = true }

[[layer]]
type = "density"
name = "Na density"
selection = { species = ["Na"] }
render = { mass_fractions = [0.50, 0.80, 0.95], inner_opacity = 0.22, outer_opacity = 0.04 }
```

Run it with:

```
mdstats-3d dump.lammpstrj --config na_lta.toml
```

Paths written inside the TOML file are resolved relative to the TOML file.

Preview the canonical scene before computing it

Use manifest-only mode:

```
mdstats-3d dump.lammpstrj \
  --config na_lta.toml \
  --manifest-only
```

This reads/resolves the source and writes the normalized scene manifest without building topology, connectivity, density, or Plotly geometry.

To print the manifest as well:

```
--print-manifest
```

This is useful for checking exactly which named layers will be present and how selections were normalized.

Configuration precedence

The rule is:

```
defaults < preset < TOML < explicit CLI
```

Layer lists are replaced, not merged.

For example, if `na_lta.toml` contains four layers but you run:

```
mdstats-3d dump.lammpstrj \
  --config na_lta.toml \
  --layer framework \
  --layer density:Na
```

only those two CLI layers are displayed.

View, visibility, and periodic display

View controls are universal and do not change the scientific products in the scene. For example:

```
mdstats-3d dump.lammpstrj \
  --config na_lta.toml \
  --camera "[111]" \
  --periodic-images 2x2x1 \
  --visible-layer "LTA framework" \
  --visible-layer "Na density"
```

Useful view options are:

```
--projection orthographic|perspective
--camera [100]|[110]|[111]|isometric
--periodic-images 2x2x1
--cell-mode reference|none
--visible-layer NAME
--show-axes
--background light|dark|transparent
--width N
--height N
```

`--visible-layer` may be repeated. Layers not listed start hidden but remain inside the self-contained HTML and can be toggled from the legend without recomputation. Plotly group-click toggles a complete named layer.

Equivalent TOML is:

```
[scene]
projection = "orthographic"
camera = "[111]"
periodic_images = "2x2x1"
visible_layers = ["LTA framework", "Na density"]
cell_mode = "reference"
```

Periodic replication is display-only. A 2x2x1 scene does not multiply density normalization, trajectory weights, connectivity occupancy, or any scientific evidence.

A layer may also set `priority = -10`, `0`, `10`, etc. in its `[[layer]]` table to control draw order. Priority is render-only; declaration order breaks ties.

Output protection

`mdstats-3d` does not overwrite an existing HTML or scene manifest by default.

To replace them deliberately:

```
--force
```

Normal execution writes both the HTML and the canonical `.scene.json` evidence.

LAMMPS input

If the dump has numeric `type` rather than `element`, supply a type map:

```
--lammps-type-map "1=Si,2=Al,3=O,4=Na"
```

If the dump does not record the unit style or physical time, also supply information such as:

```
--lammps-units metal
--lammps-timestep 0.001
```

or provide a suitable LAMMPS log with `--lammps-log`.

The CLI intentionally fails instead of guessing these quantities.

Useful resource/render controls

```
--max-threads N
--max-memory 12GiB
--wall-time-target 1200
--browser-profile compact|balanced|quality
--max-browser-faces N
```

The wall-time value is advisory; it is not a hard scientific stop.

Shared scientific dependencies and renderer-neutral output

The command surface and layer composition are universal, and raw preparation now exposes separate framework-topology, atomic-connectivity, trajectory, and density product dependencies. Omitted layer families omit their dependency key. Duplicate instances share equal keys, and concurrent requests for one scientific key execute once within a scene context.

The LTA provider still uses the qualified framework-dynamics machinery internally when that scientific owner needs joint registration/density planning. That batching is hidden behind the dependency DAG and does not make the composite scene object the scientific dependency of every layer. As of GFX3D-5, prepared layers also no longer carry the composite scene merely so the renderer can work: each adapter emits renderer-neutral primitives and the common Plotly backend only knows primitive classes.

What comes next

The common GFX3D foundation is complete through GFX3D-5. The next visualization extension is **GFX3D-RING1**, which will expose the existing ring scientific authority as an independent registered layer before cage, site, assignment, transition-path, and Markov-network layers are added.

Robust repeated plotting in 0.20.156a0

Repeated LTA plots now reuse the automatically written topology sidecar only after exact authentication against the parsed trajectory, framework connectivity policy, and framework mapping. If the trajectory geometry, frame selection/stride, mapping, or relevant cutoff policy changes, mdstats rejects the stale sidecar and rebuilds it. Use `--no-topology-cache` only when you explicitly want to force that rebuild on every run.

The cell is now a scene-level view element, so `cell_mode = "reference"` works for framework-only, trajectory-only, connectivity-only, and density-only scenes. Camera vectors and periodic replication counts are strict; invalid/fractional declarations produce an error instead of being rounded or truncated. Large periodic display requests are checked against the browser budget before replicated geometry is allocated.

For long fixed-cell framework trajectories, `0.20.156a0` adds a vectorized projected-geometry path while preserving the established frame-local algorithm as the fallback for variable-cell and periodic-multiedge cases. This is execution-only: changing worker count or taking the fast path does not change the framework scientific identity.

Faster and more diagnosable preparation in 0.20.157a0

`0.20.157a0` hardens the preparation path without changing layer science or the historical compatibility preset.

For LAMMPS dumps, positive `start`/`stop`/`stride` selection is now applied while the file is scanned. Discarded frames still have their headers counted so `source_frame_count` remains exact, but their atom tables are not tokenized and materialized. Thus a command such as

```
--stride 500
```

now reduces input memory and parsing work as well as the number of frames sent to GFX3D. Negative Python-style `start`/`stop` indices retain the full-reader fallback because their selection cannot be known until the source frame count is established.

GFX3D also emits preparation sub-stages for framework calibration, framework connectivity, topology construction, full atomic connectivity, and composite scene/density preparation. If preparation fails, the CLI reports the nested root cause rather than only a message such as `Failed to resolve GFX3D dependency 'framework_topology_product'`. The raw LTA provider latches a failed preparation, so concurrent product requests cannot retry the same expensive failed calculation several times.

Numeric LAMMPS type maps are **file-specific**. The examples in this guide are illustrative; do not copy a map unless it matches the dump that produced the trajectory. The LTA provider now warns when the mapped Si/Al framework species have implausible T-O calibration statistics or when nearly every sampled frame becomes a distinct framework topology. Those diagnostics commonly indicate a wrong type map, an actually damaged framework, or a cutoff policy that deserves inspection.

The compatibility preset intentionally remains comprehensive: it creates one density field for every present supported species. Framework Si/Al/O atoms can have very small positional spreads, causing adaptive density resolution to become much finer than the mobile-ion density. For routine ion-transport views, request only the density you need, for example:

```
mdstats-3d dump.lammpstrj \
  --lammps-units metal \
  --lammps-timestep 0.001 \
  --lammps-type-map "...file-specific mapping..." \
  --layer framework \
  --layer connectivity \
  --layer trajectory:Na \
  --layer density:Na
```

The sparse density produced by the preferred large-grid backend is now rendered directly through the sparse mesh/node-cloud path; it is no longer incorrectly sent through a dense-field `.values` interface.

Long-trajectory connectivity hardening in 0.20.158a0

`0.20.158a0` repairs the failure previously reported during `atomic_mean_graph` preparation:

```
Could not reconstruct minimum-image vectors from integer image shifts.
```

The general triclinic MIC engine now carries the integer image label through the same unimodular Minkowski transform used to determine the minimum vector. It no longer tries to infer that integer label afterward from a floating-point `inv(cell)` reconstruction, which was numerically fragile for ill-conditioned lattice representations.

Long fixed-cell LTA connectivity also uses a faster exact path. Si-O, Al-O, and present mobile-ion/O cutoff pairs share one oxygen-centered neighbor request, the fixed periodic cell-list metric stencil is cached by exact cell/cutoff identity, and framework-only connectivity is projected from the broader hysteretic graph when both products are requested. This projection is qualified against a separate direct framework computation: state digests, frame-state IDs, and transitions are identical.

The canonical-state reuse cache is bounded. Highly fragmented trajectories therefore cannot retain an ever-growing set of raw Python edge tuples just as cache keys. The atomic mean graph also has a certified periodic-mean fast path: compact distributions use a unique single-start solution only after a strong-convexity certificate; ambiguous/mobile distributions continue to use the exact multi-start fallback.

On the supplied 10,001-frame Na-LTA dump using the likely mapping `1=Al,2=Na,3=O,4=Si`, a full framework + connectivity + Na-trajectory source preparation completed without the former MIC failure. Atomic connectivity resolved in about 41.5 s and framework projection in about 1.3 s in the qualification environment. A 400-frame cold-connectivity comparison using the same scientific definition measured about 6.59 s in `0.20.157a0` versus 1.59 s in `0.20.158a0`, with identical connectivity identity.

The current connectivity fold remains deliberately serial by default. The existing frame-threaded candidate path was benchmarked on this workload and was slower because hysteresis is stateful and the Python graph fold/worker materialization cost outweighs thread-level neighbor-search gains. HARDEN3 therefore improves the algorithm and memory scaling instead of enabling a slower thread mode merely to increase CPU utilization.

Density realization hardening in 0.20.159a0

`0.20.159a0` addresses runs that appeared to stop at:

```
density_realization [0/4 fields]: constructing independent density fields through the PAR-DENS3
scheduler
```

The scheduler was generally alive; the large adaptive `local_sparse` fields were executing direct sparse tiles on one CPU core per field because the low-level direct executor did not consume the cooperative scheduler lease. The revised executor parallelizes target-coordinate and packed-lookup work inside the existing approved pair chunk and preserves the historical canonical reduction order exactly.

Long runs now show scheduler admission and kernel progress, for example:

```
density_scheduler_task [0/4 fields]: started atomic-density-2; backend=local_sparse; workers=1; peak=...
MiB
hybrid_sparse_realization: atomic-density-2: realizing ... direct tile(s) and ... FFT tile(s); direct
sparse work is CPU execution
hybrid_direct_realization [.../... pairs]: atomic-density-2: CPU direct sparse convolution; workers=3
```

The live worker count may increase after sibling fields finish because PAR-DENS returns those CPU tokens to the remaining tasks. Direct sparse tiles are CPU kernels, so zero GPU utilization is normal when an approved field contains no FFT tiles. GPU execution remains an optional execution path for FFT tiles only.

The historical four-species compatibility preset is unchanged. For mobile-ion analysis, explicitly selecting `density:Na`, `density:Li`, or `density:K` still avoids spending time on framework-species volumetric densities when they are not scientifically needed.